Relational Database Systems –

COSC265

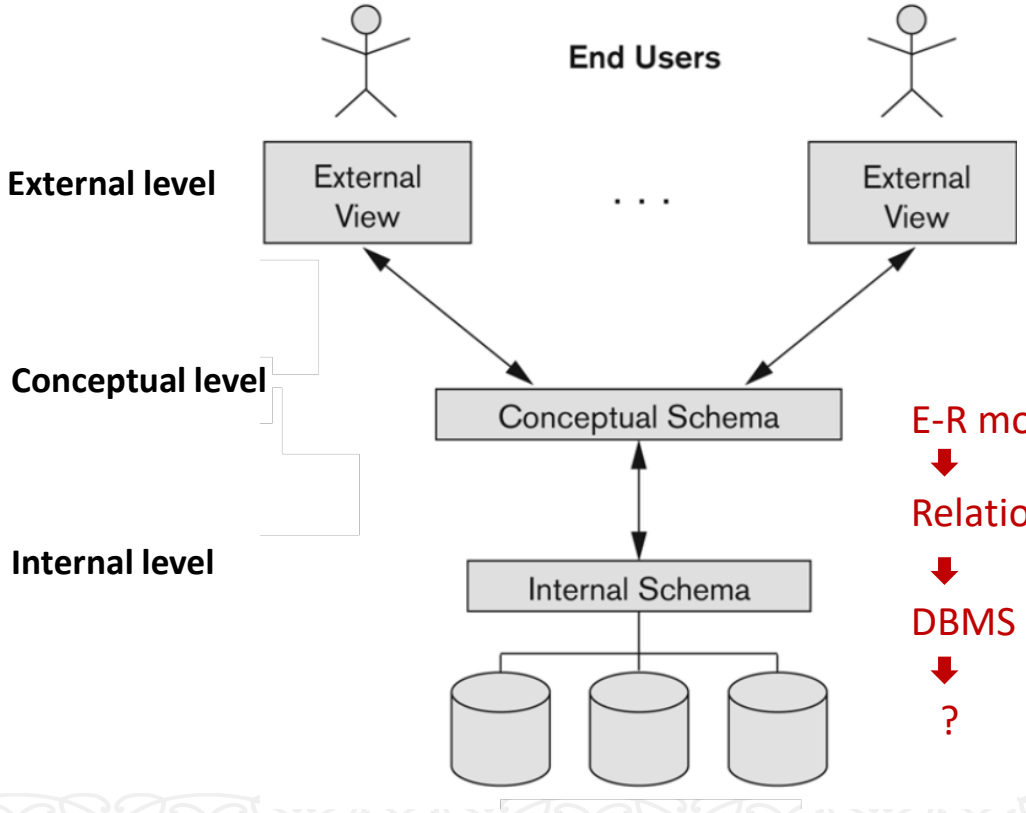
Lectures 23 & 24 – Physical Level



Miguel Morales

UC UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

Overview *(a quick one)*



E-R model

⬇

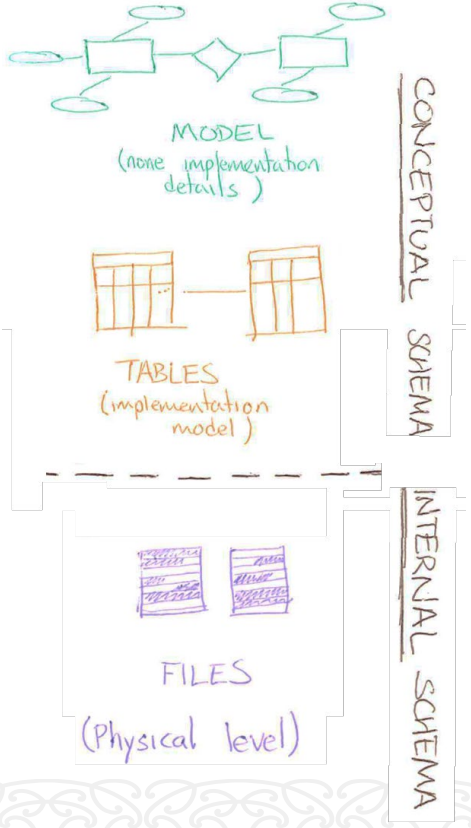
Relational Model

⬇

DBMS + SQL + …

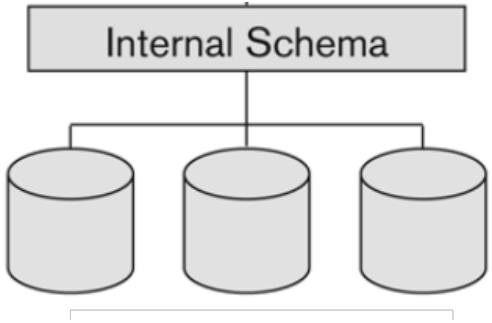
⬇

?



Internal Level

- Memory hierarchy
- Disk space management
- Record and File organization
- Redundant Array of Independent Disks (RAID)
- Indexing
- Physical level in Oracle



Why should we care about the Internal Level?

- How is the data stored in the persistence layer?
- How can we speed up access to data?

Who is in charge of this? Is it the DBD?

Database Performance

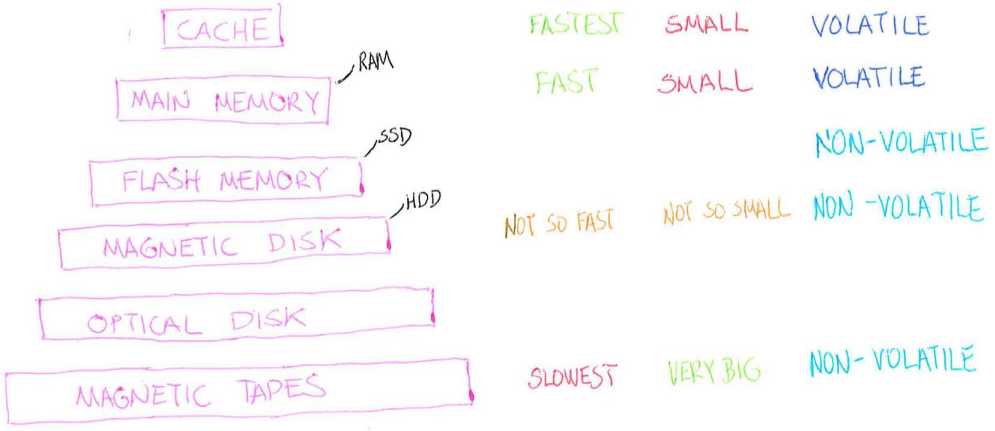
- **Database performance** can be determined by many factors:
  - Memory Hierarchy
    - Secondary storage
  - Memory management
    - Buffering and Cache
  - Indexing strategies
  - Query optimization
  - Network factors
    - Traffic/volume, location
  - Concurrency
  - …

How can we influence these factors?

- Data Types selection (DDL)
- Degree of normalization
- Query formulation (DML)
- Constraints and Triggers
- Indexes
- Data distribution
- What we store (or don't)
- …

Memory Hierarchy

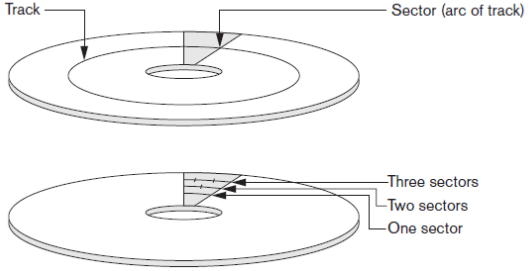
- **Primary**
  - Fast access
  - Limited capacity
  - Volatile (transient data)
  - **E.g., cache and main memory**

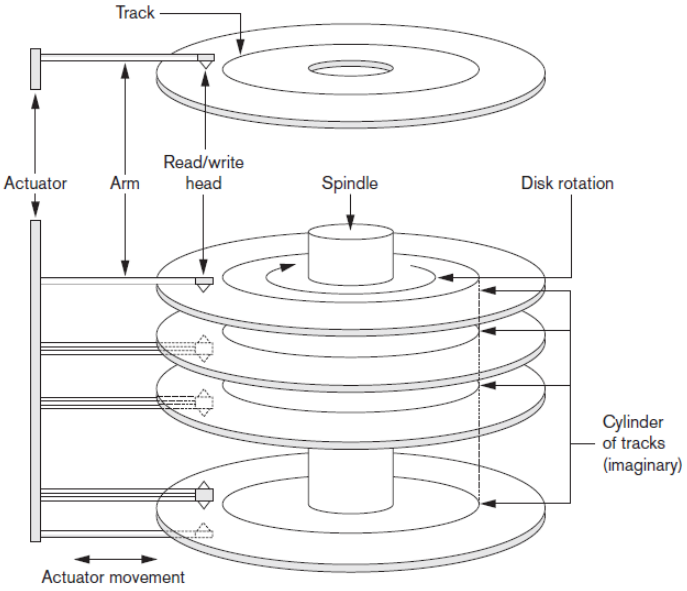


- **Secondary** and tertiary
  - High capacity
  - Low cost
  - Non-volatile (persistent data)
  - **E.g., magnetic disks, solid-state drives, tapes**

Secondary Storage Devices

- A **Hard Disk Drive**:
  - Concentric circles called tracks
  - Tracks are divided into blocks or sectors
  - Read/write head





  - A physical disk block **address** consists of:
    - A cylinder number (imaginary collection of tracks of the same radius from all recorded surfaces) – *the suburb*
    - The track number or surface number (within the cylinder) – *the street*
    - and block number (within the track) – *the actual house it lives*

Why do you think addresses are relevant?

What if we have (or need) more than one disk?

File Organization

- **File Organization** determines:
  - How records are physically **stored** (placed) on the disk
  - How records are **accessed**

- Therefore, the **DBMS** needs to **interact with** the **OS** and the **file system** to read/write physical blocks between secondary and primary memory

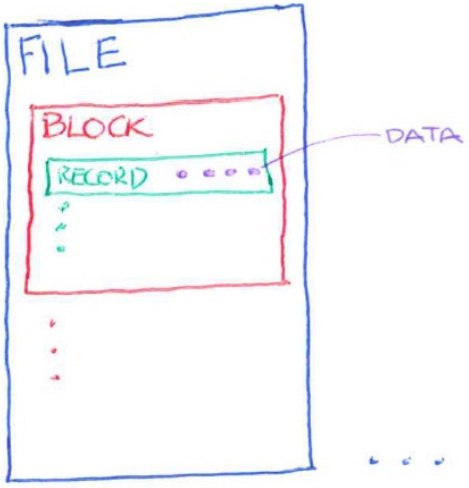
Why do DBMSs not store all data in (primary) memory?

- In-memory databases are database systems that rely (primarily) on main memory for data storage
  - Pros: faster access
  - Cons: loss of persistence if the system crashes
  - Examples: H2 and SQLite

When does it make sense to use this type of database?

File Organization

- A **file** is a *collection* of blocks
- A **block** is a *sequence* of records
- A **record** is a *set* of values
- Values are stored in **fields** of a particular type

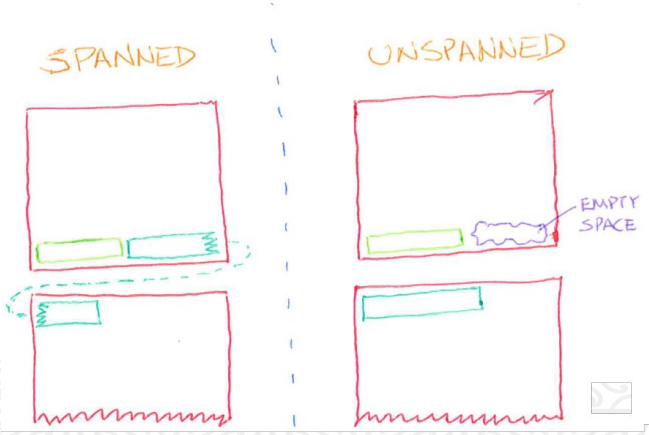


- A block can have **fixed-length** or **variable-length** records
- Fields themselves may be fixed-length or variable-length too

How can we map these Physical-level related terms to what we know about Database Design?

File Organization

- A file **header** includes information that describes the file
  - Field names and their data types
  - The addresses of the file blocks on disk
- **Blocking Factor** (bfr): refers to the number of records that can be stored per block
  - There may be empty space in a block if an integral number of records do not fit in one block.
- File records can be
  - **Unspanned:** no record can span two blocks
  - **Spanned:** a record can be stored in more than one block



Operation on Files

UC
UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

- **OPEN** – Reads the file; a pointer refers to the current record
- **FIND** – Searches for the first record that satisfies a certain condition and makes it the current record.
- **FIND_NEXT** – Searches for the next record (from the current record) that satisfies a certain condition and makes it the current record
- **READ** – Reads the current file record into a program variable
- **INSERT** – Inserts a new record into the file and makes it the current file record
- **DELETE** – Removes the current file record, usually by marking it to indicate that it is no longer valid.
- **MODIFY** – Changes the values of some fields of the current record.
- **CLOSE** – Terminates access to the file.
- **READ_ORDERED** – Reads the file blocks in order of a specific field of the file.

Types of Files

- **Unordered** Files
  - Also called a heap or a pile file
  - **Efficient insertion,** as new records are inserted at the end of the file
  - **Linear search can be expensive**, as it requires going through the file records

- **Ordered** Files
  - Records are sorted by an **ordering field**
  - **Expensive update**, as records must be inserted in the right order
  - **Binary search** can be used to search for a record on its ordering field value

What if I need to search a field other than the ordering field?

  - The sorting should be done in the primary memory, involving a lot of reads, making it expensive

Types of Files

- What type of file is it?
- Does it have an ordering field?
- Describe the process the DBMS would follow to insert a record for “Adams, Steven”



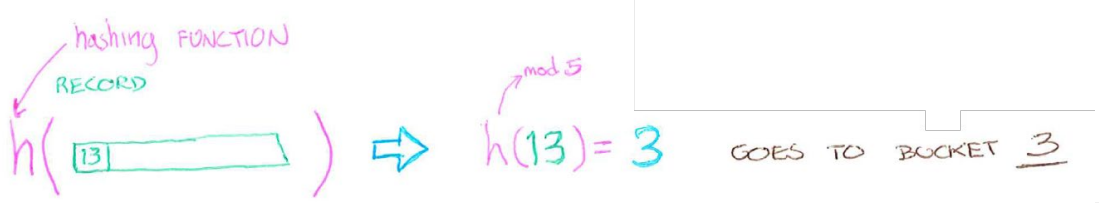
	NAME	SSN	BIRTHDATE	JOB	SALARY	SEX
block 1	Aaron, Ed					
	Abbott, Diane					
	⋮					
	Acosta, Marc					
block 2	Adams, John					
	Adams, Robin					
	⋮					
	Akers, Jan					
block 3	Alexander, Ed					
	Alfred, Bob					
	⋮					
	Allen, Sam					
block 4	Allen, Troy					
	Anders, Keith					
	⋮					
	Anderson, Rob					
block 5	Anderson, Zach					
	Angeli, Joe					
	⋮					
	Archer, Sue					
block 6	Arnold, Mack					
	Arnold, Steven					
	⋮					
	Atkins, Timothy					
	⋮					
block n –1	Wong, James					
	Wood, Donald					
	⋮					
	Woods, Manny					
block n	Wright, Pam					
	Wyatt, Charles					
	⋮					
	Zimmer, Byron					

Types of Files

- **Hashed** File
  - The file blocks are divided into **equal-sized buckets**
  - Typically, a bucket corresponds to one [or a fixed number of] disk block(s)

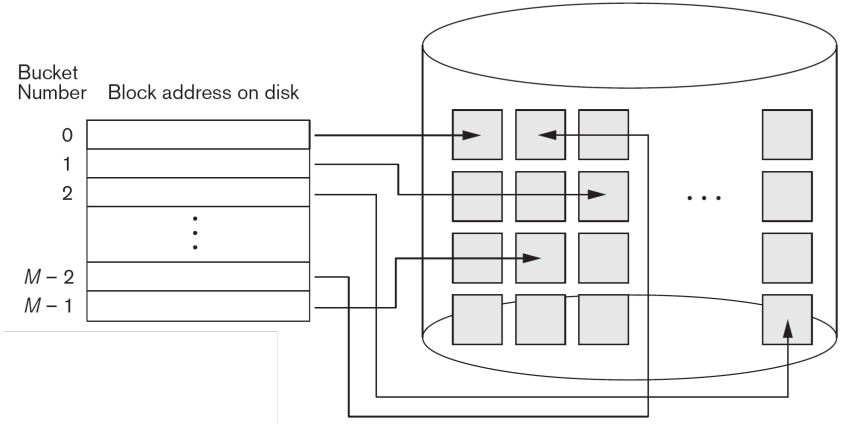


  - The hash key is **computed for each file record**
  - The record with hash key value *k* is stored in bucket *i*, where $h(k) = i$ is the hashing function



Hashed Files

- **A block is used to store the block addresses** for each of the possible values of the hashing function
- Search is very efficient on the hash key
- **Collisions** occur when a new record hashes to a bucket that is already full
- An *overflow bucket* is kept for storing such records



Collision Resolution Methods

UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

- **Open addressing:**
  - Proceeding from the occupied position specified by the hash address, the program checks the subsequent positions in order until an unused (empty) position is found.

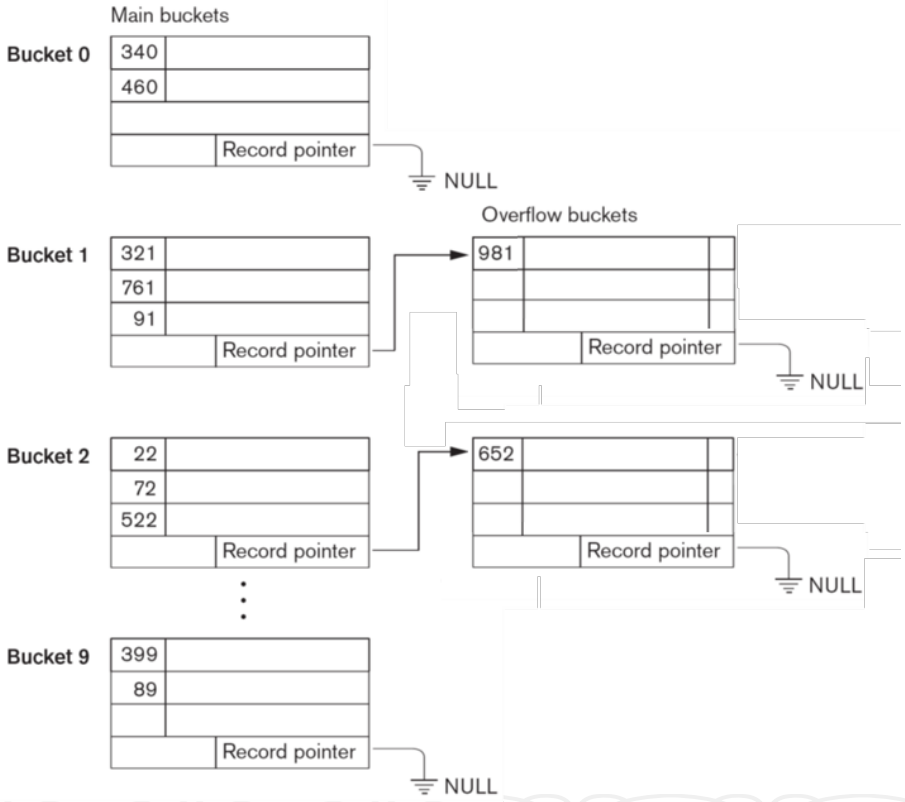
    Next available bucket
- **Chaining:**
  - Various overflow locations are kept, usually by extending the array with a number of overflow positions. In addition, each record location has a pointer field. A collision is resolved by placing the new record in an unused overflow location and setting the pointer of the occupied hash address location to the address of that overflow location.

    Overflow bucket
- **Multiple hashing:**
  - It applies a second hash function if the first results in a collision. If another collision results, it uses open addressing or applies a third hash function and then uses open addressing if needed.

    Multiple hashing functions

Collision Resolution Methods

- Assume that records 490 and 1510 will be inserted
- Where will these be inserted if we use:
  - Open Addressing
  - Chaining



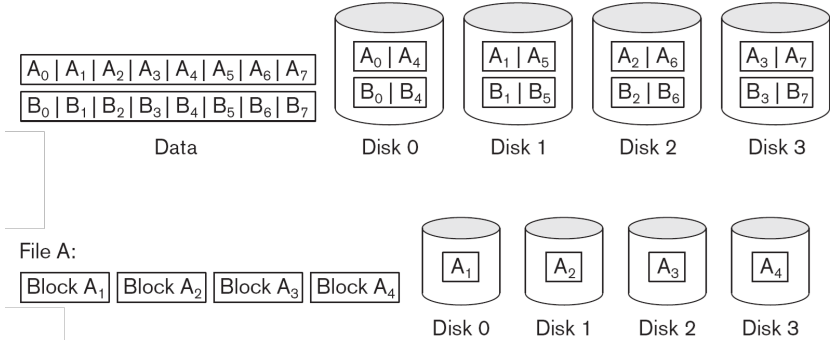
Hashed Files

UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

- To reduce overflow records, a hash file is typically **kept 70-80% full**
- The hash function should **distribute** the **records uniformly** among the buckets
- **Ordered access** on the hash key **is** quite **inefficient**, as it requires sorting the records

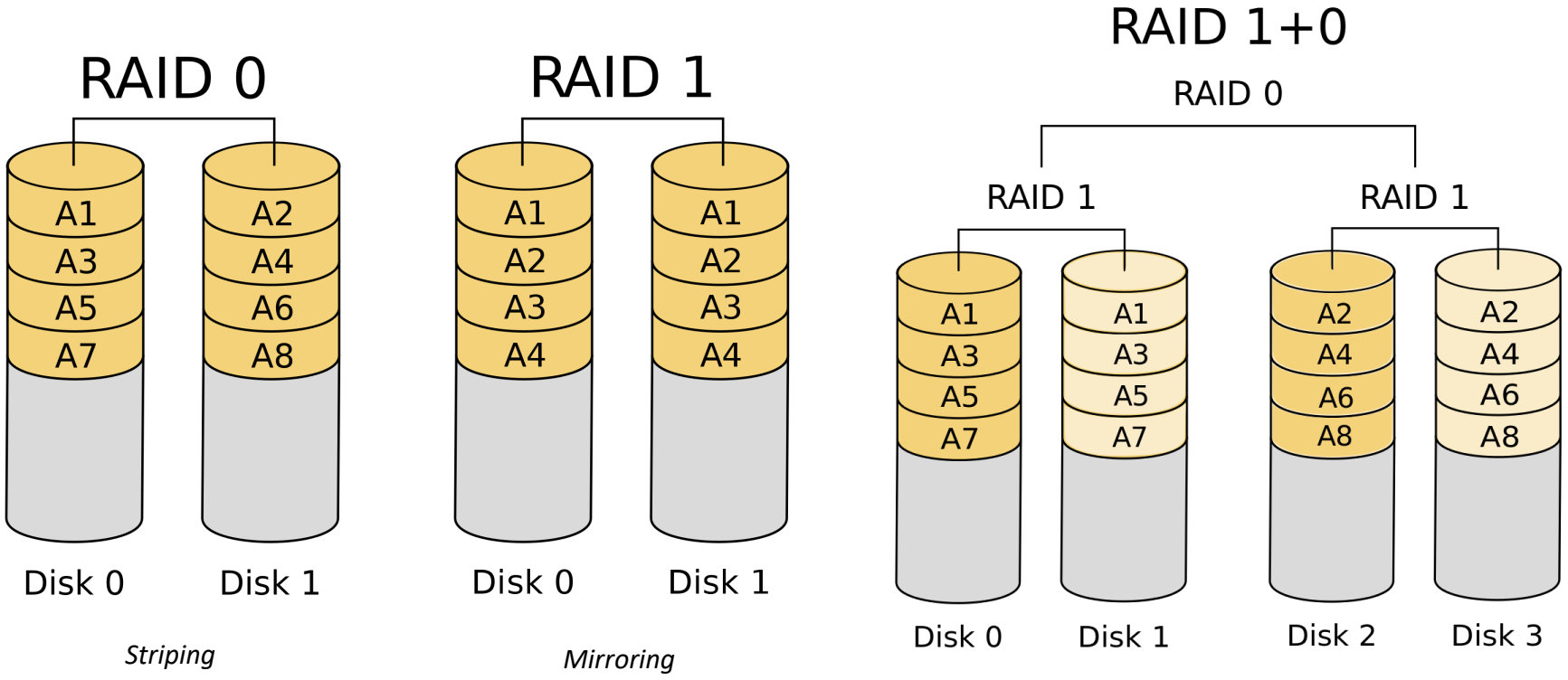
RAID

- Redundant Arrays of Independent Disks (RAID)
  - For improving **performance** and **reliability**
  - **Data striping** is a storage technique that splits a file into smaller segments and spreads them across multiple physical storage devices to improve performance

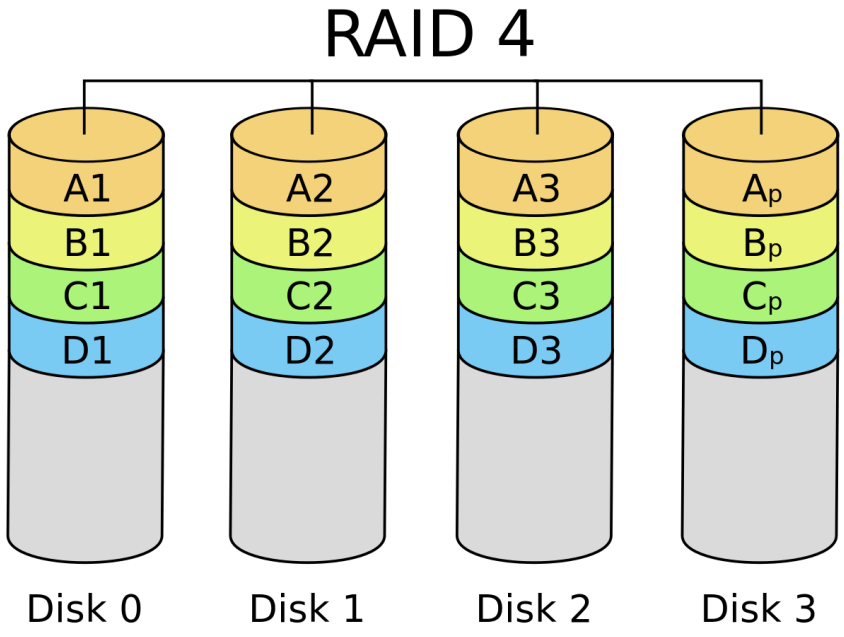


  - *A disk array with data distributed over all disks, appearing to the user as a single large disk*

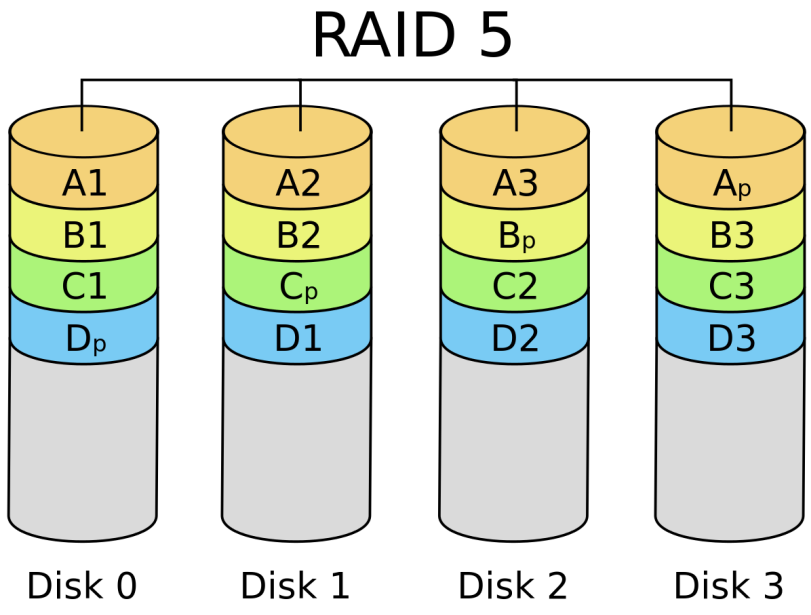
RAID


RAID 0
A1
A3
A5
A7
A2
A4
A6
A8
Disk 0
Disk 1
Striping
RAID 1
A1
A2
A3
A4
A1
A2
A3
A4
Disk 0
Disk 1
Mirroring
RAID 1+0
RAID 0
RAID 1
RAID 1
A1
A3
A5
A7
A1
A3
A5
A7
A2
A4
A6
A8
A2
A4
A6
A8
Disk 0
Disk 1
Disk 2
Disk 3

RAID

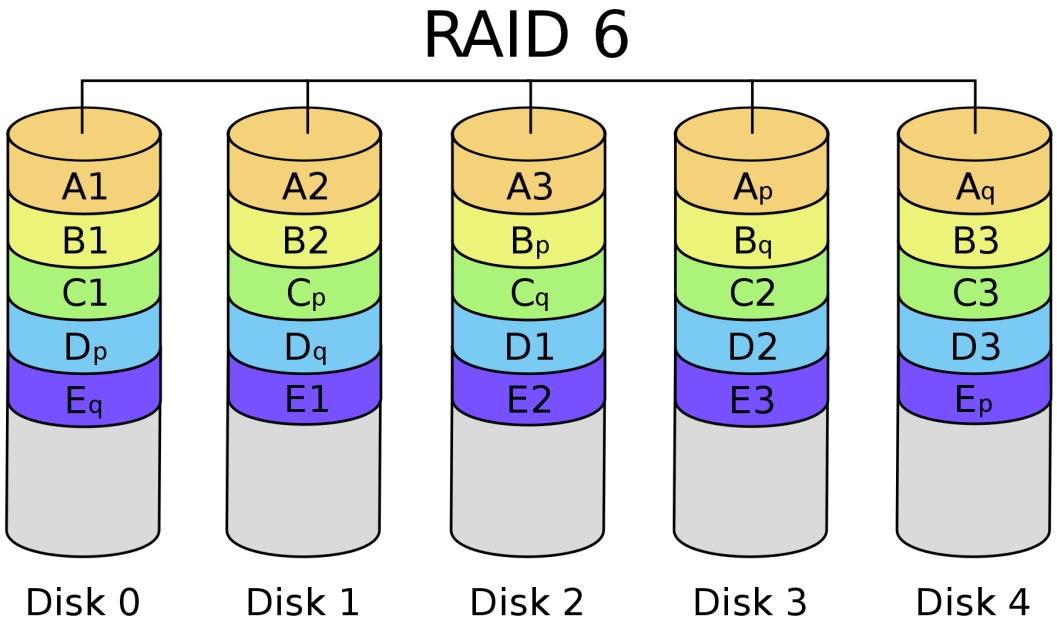


Dedicated parity



Distributed parity

RAID



Dual parity

Parity Check for Error Correction

UC
UNIVERSITY OF CANTERBURY
Te Whare Wānanga o Waitaha
CHRISTCHURCH NEW ZEALAND

- **Hamming Code** (bit-level) → RAID 4 and 5
  - Adds parity **bits** at specific positions
  - Overlapping parity checks locate the erroneous bit
  - **Corrects 1-bit errors** and detects certain multi-bit errors
  - Best suited to isolated bit errors
  - *It uses basic **XOR** calculations*

- **Reed-Solomon Code** (block/symbol-level) → RAID 6
  - Treats data as **symbols** rather than individual bits
  - Adds redundant symbols using polynomial-based encoding
  - Can **recover from multiple corrupted symbols**
  - Particularly effective when many consecutive bits are damaged
  - *It uses advanced abstract algebra (**Galois Fields**) to construct a curve through the data points*

RAID

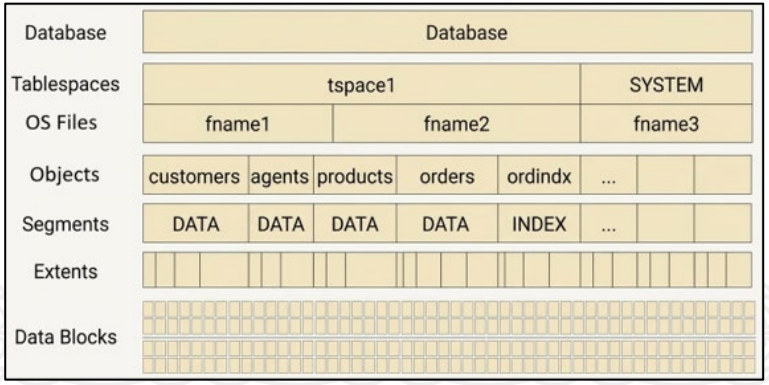
Level	Storage Efficiency	Fault Tolerance (max drive failures)	Retrieval Efficiency (read speed)	Write Efficiency (write speed)	Minimum Drives
RAID 0	N = number of disks	0	**Excellent** (Fastest)	**Excellent** (Fastest)	2
RAID 1	N/2	N-1	**Good** (Can read from multiple mirrors)	**Poor** (Limited to a single drive)	2
RAID 4	(N-1)/N	1	**Good** (Parallel reads across data drives)	**Poor** (Dedicated parity drive creates a major bottleneck)	3
RAID 5	(N-1)/N	1	**Very Good** (Parallel reads across all drives)	**Fair** (Write penalty for parity)	3
RAID 6	(N-2)/N	2	**Very Good** (Parallel reads across all drives)	**Poor** (Heavy write penalty for dual parity)	4
RAID 1+0	(N*C)/2	1 to N/2	**Excellent** (Parallel reads across all drives)	**Good** (No parity calculation required; twice as fast as RAID 1)	4

Physical Level in Oracle

- **Database** – Logical Level
- **Tablespaces** and **OS Files** – Logical and Physical Level
  - It shows the relationship between Oracle's logical storage structures and the physical OS files

- **Tablespaces** – Logical Level
  - These are logical storage containers
  - An Oracle database can have more than one tablespace (SYSTEM and User-defined ones)
  - A tablespace can span across multiple physical files
  - A single file can only belong to one tablespace

- **OS Files** – Physical Level
  - These are the physical data files



Physical Level in Oracle

- **Objects** – Logical Level
  - The actual schema objects that users interact with
- **Segments** – Logical Level
  - A segment is a set of extents allocated for a specific logical structure
  - It is automatically allocated when the object is created
  - Every object has its own DATA segment
- **Extents** – Logical-Physical Level
  - It is a specific number of contiguous data blocks
- **Data Blocks** – Physical Level
  - Represent the smallest physical unit of I/O storage in an Oracle database

